Extending LMCache's Prefix-Cache Eviction Policy: Cost-Awareness and Windowed Admission Control

Design, Evaluation, and Ablation Report

Project: LMCache -- prefix-cache eviction policy extension

Repository: LMCache (branch: cost-aware-policy)

Author: Aya Neeman

Date: July 31, 2026

Abstract. LMCache's storage backends select which cached KV chunks to evict using simple recency/frequency baselines, blind to recompute-cost heterogeneity and to the difference between a low- and high-value newcomer. This report evaluates three extensions: (A) a cost- and frequency-aware scoring policy, (B) a TinyLFU-style admission-control wrapper that can reject low-value newcomers outright, and (C) a windowed variant of (B) removing a correctness limitation found while evaluating it. An external review of an earlier draft surfaced five methodological problems -- a frequency double-count, a wall-clock dependency that made the cost-aware recency term inert inside a sub-second benchmark run, an ablation that never exercised its own parameter, single-run headline numbers, and a statistical comparison that ignored a paired sampling design -- all fixed, with every number here reflecting the corrected, fully re-run pipeline. Cost-awareness is now a real, statistically supported improvement on synthetic and real traffic alike (previously the apparent weakest baseline on real data, a wall-clock artifact); admission control remains the strongest general-purpose improvement, with paired significance at nearly every cell tested; the windowed variant trades measurable upside for eliminating a catastrophic freeze mode. One real regression is confirmed and reported as such.

1. Introduction

LMCache accelerates LLM inference by caching key-value (KV) tensors for previously seen token prefixes, so a later request that shares a prefix (a repeated system prompt, an earlier turn of the same conversation, a common few-shot template) can reuse cached KV instead of recomputing it. Under memory pressure, a cache-eviction policy decides which cached chunks to discard to make room. LMCache ships this decision behind a small, swappable BaseCachePolicy interface (lmcache/v1/storage_backend/cache_policy/), with four baseline implementations available out of the box: LRU, LFU, FIFO, and MRU.

These four baselines share a blind spot: they rank cached chunks purely by recency or by raw access count, and they have no way to refuse a newcomer -- eviction always makes room for whatever arrives next, regardless of whether the newcomer is likely to be reused. This motivates two independent ideas from the broader caching literature: cost-aware eviction (scoring candidates by a combination of expected reuse and recompute cost, in the tradition of GreedyDual-Size-style web-cache policies) and frequency-gated admission control (rejecting a newcomer outright instead of always evicting an incumbent, in the tradition of the TinyLFU / Window-TinyLFU design used by Caffeine, a widely deployed JVM caching library). This project ports both ideas into LMCache's cache_policy abstraction, builds a CPU-only benchmark harness that replays the real policy call sequence a storage backend makes, and evaluates the resulting policies on both synthetic workloads and a real multi-turn conversation corpus (ShareGPT).

An earlier draft of this evaluation reached broadly similar qualitative conclusions but rested on a benchmarking pipeline with real bugs -- caught by an external review, not by this project's own testing. Section 6.4 documents all five and their effect on the numbers; the short version is that every result in this report was re-run after fixing them, so the numbers here supersede any earlier draft, including this project's own design docs (which now carry an errata notice pointing back to this report). The central question this report answers is not just "does the new policy beat the baseline on one benchmark," but whether the improvement is general and statistically real -- does it hold across cache sizes, does it hold on real traffic under a sampling design that respects how that traffic was generated, and does it introduce any new failure modes the baseline didn't have.

2. Extension Design

2.1 Baseline

BaseCachePolicy defines the contract every policy implements: init_mutable_mapping (construct the cache's backing dict), update_on_hit / update_on_put (record an access or insertion), get_evict_candidates (rank and return keys to remove under pressure), and update_on_force_evict (cleanup hook). LRU, LFU, FIFO, and MRU each implement this with O(1)-amortized bookkeeping and serve as both the production defaults and the baseline this report measures every extension against.

2.2 Direction A -- CostAwareEvictionPolicy

Scores each resident chunk by combining three signals: an EWMA-smoothed cost density (observed/estimated recompute cost divided by the chunk's memory footprint), a reciprocal (hyperbolic) recency decay -- score is divided by (1 + age/half_life_seconds), not multiplied by an exponential falloff --

and a log-dampened access-frequency term. The frequency term was added specifically to fix an initial cost-only version that lost to plain LRU/LFU on real ShareGPT data: with no frequency signal, cost-density alone could keep an expensive chunk resident indefinitely even after it stopped being reused. get_evict_candidates ranks by this score ascending (lowest score evicted first).

Bug found by review, fixed here: every recency computation used to call time.monotonic() directly. A benchmark run replays its entire request sequence in well under one second, while the default half_life_seconds is 60.0 -- age_seconds was therefore always approximately zero regardless of access order, making the recency term's real effect invisible in every result the original evaluation reported, and non-reproducible across machines of different speeds (Section 6.4). The class now takes an injected clock (defaulting to time.monotonic for production use); the benchmark simulator injects a deterministic logical clock -- one tick per simulated request -- automatically.

2.3 Direction B -- AdmissionControlledPolicy

Wraps any inner policy (LRU, LFU, FIFO, MRU, or CostAware) and adds one new hook, should_admit(key, cache_dict), called only when the cache is already full. It maintains its own decaying frequency sketch (a plain dict with periodic halving, not a real Count-Min Sketch) and admits a newcomer only if its estimated frequency exceeds the coldest currently-resident key's estimate; otherwise the newcomer is rejected and the incumbent keeps its slot. get_cache_policy("ADMISSION_<INNER>") composes with any registered policy name.

Two bugs were caught while building this as a real, tested class (rather than trusting the prototype that motivated it): (1) the first version didn't record a frequency observation for a rejected key, so a key that lost its first admission bid could never win a later one -- a permanent lockout; (2) the first version speculatively called the inner policy's get_evict_candidates just to compare frequencies, which corrupted LFUCachePolicy's internal bookkeeping when the speculative peek was discarded. Both are fixed in the shipped class. A third bug, found by the external review rather than by this project's own testing, is more subtle and is documented in full in Section 6.4: should_admit incremented the frequency sketch once, and then the caller's own follow-up call (update_on_put_with_metadata, made immediately afterward whenever admission succeeds) incremented it a second time for the same request -- every admitted key's frequency was silently double-counted relative to a rejected key or a fill-phase insertion. Fixed by tracking, per class instance, whether the immediately-preceding should_admit call already recorded this exact key, and skipping the redundant increment when it did.

2.4 Direction C -- WindowedAdmissionControlledPolicy

Evaluating Direction B to the same depth as Direction A surfaced a real design limitation: should_admit's strict greater-than comparison always favors the incumbent on a tie, which is exactly right under heavy eviction pressure but has two failure modes elsewhere -- a real hit-rate regression under generously sized, low-pressure caches, and a permanent, silent freeze under purely one-shot traffic. WindowedAdmissionControlledPolicy fixes both by construction: new keys always enter a small, bounded, always-admits window; only when the window overflows is its oldest member evaluated -- promoted into the frequency-gated main region if it clears a promotion threshold, or queued for a real eviction otherwise. Because the window never rejects, the freeze failure mode is structurally unreachable. Kept as a second, independently selectable policy rather than a rewrite of Direction B, so the two remain directly comparable (Section 4).

3. Experimental Setup

3.1 Tooling

lmcache/tools/cache_policy_bench/runner.py implements a CPU-only simulator (_PolicyCache) that drives a real policy object through the exact call sequence a storage backend makes on every request. A CostModel maps hit-prefix length and recompute-token count to a modeled latency (no GPU or running model required). run_workload injects a deterministic logical clock into any CostAwareEvictionPolicy it constructs (Section 2.2) and reports diagnostic fields (e.g. sketch_halvings_triggered) whenever a policy exposes them, so an ablation script can verify a parameter actually had the intended effect rather than assuming it did.

3.2 Workloads

- repetitive_short -- small vocabulary, high reuse density.
- novel_long -- long, unique documents touched exactly once; the purely one-shot stress case for admission control.
- mixed_zipfian -- Zipf-distributed popularity over a prefix pool; skew strength (zipf_s) is a swept parameter.
- multi_round_chat -- simulated multi-turn sessions with growing shared prefixes. Its seed argument is unused by design (the generator is fully deterministic); every result from this workload in this report is labeled a single-run case study, not statistical evidence -- see Section 3.3.
- Real ShareGPT corpus -- roughly 35,000 real multi-turn conversations, adapted into the same Request shape the synthetic generators produce.

3.3 Statistical method

Every headline claim in this report is backed by repeated, independently seeded runs, not a single reading. For the three seed-capable synthetic workloads (repetitive_short, novel_long, mixed_zipfian), each (policy, cache-size) cell is run across 10 independent seeds. For the real ShareGPT corpus, requests_from_conversations draws max_conversations conversations via random.sample -- sampling without replacement -- which is repeated subsampling, not a corpus bootstrap; each (policy, scale, cache-size) cell is run across 6 repeats.

Crucially, every policy at a given seed/repeat index replays the identical generated workload instance (same seed -> same random.sample draw), so per-repeat readings are paired across policies, not independent. Comparing two policies by checking whether their independently computed confidence intervals overlap discards that pairing and understates the evidence for a real difference -- Section 4.1 shows a concrete case where this matters. The statistically correct comparison, used for every "policy X beats policy Y" claim in this report, is the per-repeat difference's own bootstrap CI (paired_bootstrap_ci_diff) plus an exact paired sign test (paired_sign_test) as a distribution-free cross-check -- both in benchmarks/cache_policy/stats.py.

3.4 Parameters swept

- Cache size: 50, 100, 200 MiB (synthetic and real-data sweeps).
- Corpus scale: 500, 2,000, 5,000 conversations (real-data only).
- Zipf skew: zipf_s in {0.6, 1.2, 2.0} (mild to extreme; single run per point -- an illustrative pattern check, not a confidence-interval claim).

- halve_every (frequency-sketch decay window): 5,000 / 20,000 (shipped default) / 80,000, on a workload sized so every value tested triggers multiple halving passes (Section 5.2).
- window_capacity (Direction C): 5 / 20 (default) / 80.
- promotion_threshold (Direction C): 1 / 2 (default) / 4.

4. Results

4.1 Synthetic cache-size sweep: vanilla vs. extended

Figure 1 sweeps mean hit rate (+/- 95% CI across 10 seeds) across all three cache sizes for the three seed-capable workloads and five representative policies. Table 1 gives the exact mixed_zipfian numbers, the workload with the most eviction pressure.

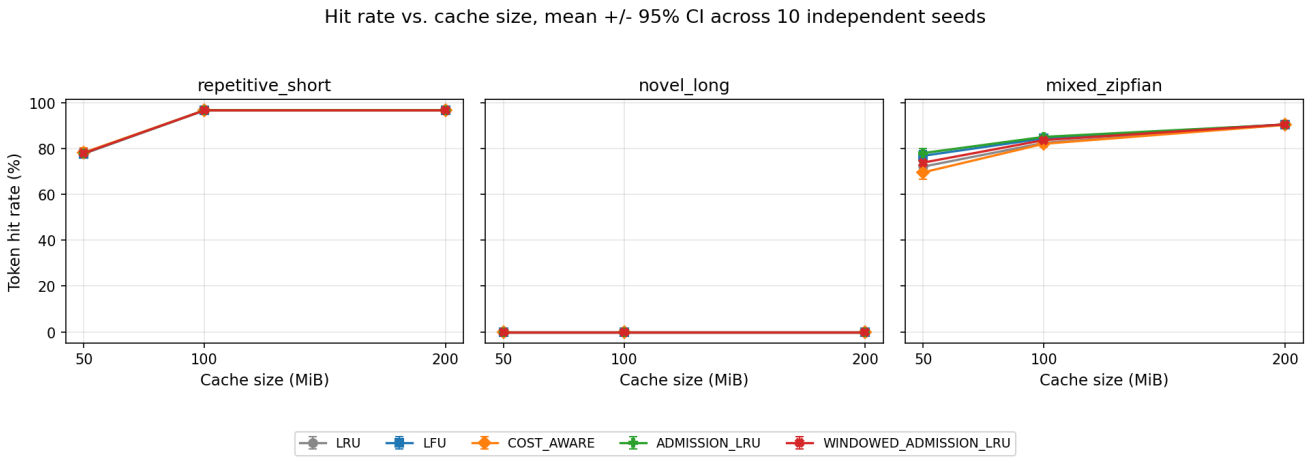

Figure 1. Token hit rate vs. cache size, mean +/- 95% CI across 10 independent seeds. novel_long is 0% for every policy by construction (no chunk is ever touched twice).

Table 1. mixed_zipfian mean hit rate/evictions/p95, 50 and 100 MiB, 10 independent seeds.

Cache	Policy	Hit rate (mean, 95% CI)	Mean evictions	Mean p95
50 MiB	LRU (baseline)	72.1% [69.2,74.9]	3,436	35.8ms
50 MiB	LFU	76.9% [74.2,79.2]	2,813	33.8ms
50 MiB	COST_AWARE	69.6% [66.6,72.5]	3,766	30.8ms
50 MiB	ADMISSION_LRU	78.0% [76.0,80.0]	402	33.3ms
50 MiB	WINDOWED_ADMISSION_LRU	73.9% [71.2,76.3]	3,204	35.3ms
100 MiB	LRU (baseline)	82.5% [80.9,84.1]	1,893	30.7ms
100 MiB	LFU	84.4% [82.8,86.0]	1,633	30.2ms
100 MiB	COST_AWARE	82.1% [80.5,83.8]	1,941	25.2ms
100 MiB	ADMISSION_LRU	85.1% [83.8,86.4]	359	30.2ms
100 MiB	WINDOWED_ADMISSION_LRU	83.7% [82.2,85.2]	1,731	30.2ms

At 100 MiB, ADMISSION_LRU's and LRU's descriptive 95% CIs overlap (83.8-84.1) -- reading Table 1 alone, a naive comparison would call this "not clearly different." It is different: because every policy at a given seed replays the identical generated request sequence, the correct comparison is the paired per-seed difference, not two independent intervals. Computed from the exact same 10 runs: ADMISSION_LRU beats LRU by a mean of +2.63 percentage points [+2.16,+3.05], exact sign test p=0.0020 (all 10 seeds favor ADMISSION_LRU) -- the strongest significance this design of test can report at n=10. COST_AWARE, whose descriptive CI also overlaps LRU's at 100 MiB, is

paired-significantly *worse* by a small but consistent -0.37pp [-0.57,-0.17]. This is a direct, worked demonstration of why Section 3.3's paired methodology matters: two policies whose own CIs overlap can still have a real, statistically supported difference once the shared sampling structure is used correctly, in either direction.

At 50 MiB the paired effect is larger and unambiguous in the raw table too: ADMISSION_LRU +5.90pp [+5.14,+6.84], WINDOWED_ADMISSION_LRU +1.73pp [+1.44,+2.04], both sign_p=0.0020; COST_AWARE is paired-significantly worse, -2.51pp [-3.04,-2.00]. At 200 MiB (not shown in Table 1) every policy converges to within 0.1-0.2pp of LRU and only COST_AWARE's small residual difference remains significant -- consistent with the established pattern throughout this evaluation that policy choice stops mattering once the working set comfortably fits in cache.

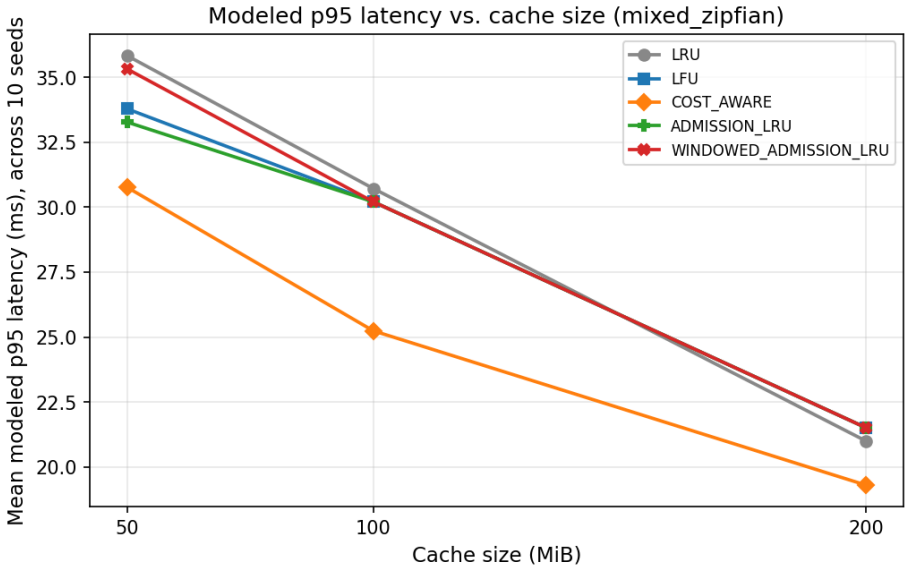

Figure 2. Mean modeled p95 latency vs. cache size, mixed_zipfian, across the same 10 seeds as Figure 1 -- lower is better.

4.2 Why hit rate improves: evictions vs. rejections

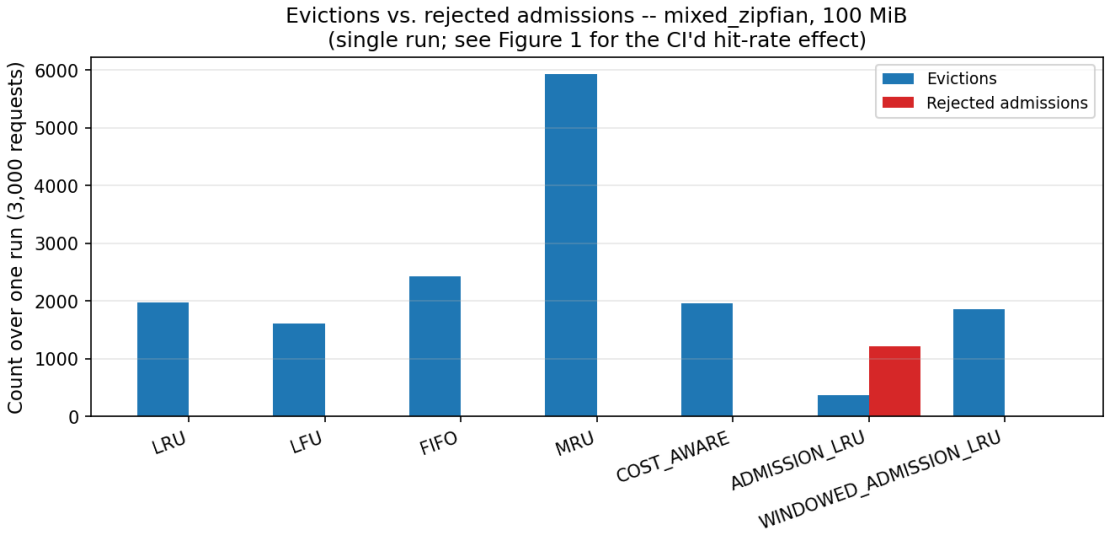

Figure 3. Evictions vs. rejected admissions, mixed_zipfian, 100 MiB, one illustrative run (mechanism only -- see Figure 1 for the CI'd hit-rate effect).

Figure 3 makes the mechanism concrete: ADMISSION_LRU's evictions collapse relative to LRU's because most of what would have been an eviction becomes an outright rejection instead -- the

newcomer that would have displaced a warmer incumbent is simply never let in. WINDOWED_ADMISSION_LRU's rejected-admission count is always exactly zero by design (Section 2.4): churn is entirely expressed as evictions from the window, at a rate closer to plain LRU's.

4.3 Real-data validation (ShareGPT)

Figure 4 shows the paired hit-rate difference against LRU, with 95% CI, at 200 MiB across three corpus scales -- every bar is a paired_bootstrap_ci_diff over the 6 shared-subsample repeats; a bar whose error bar excludes zero is a statistically supported difference from LRU, not a description of two overlapping intervals. Table 2 gives the full hit-rate grid across all three cache sizes at 500 conversations, including the paired significance verdict.

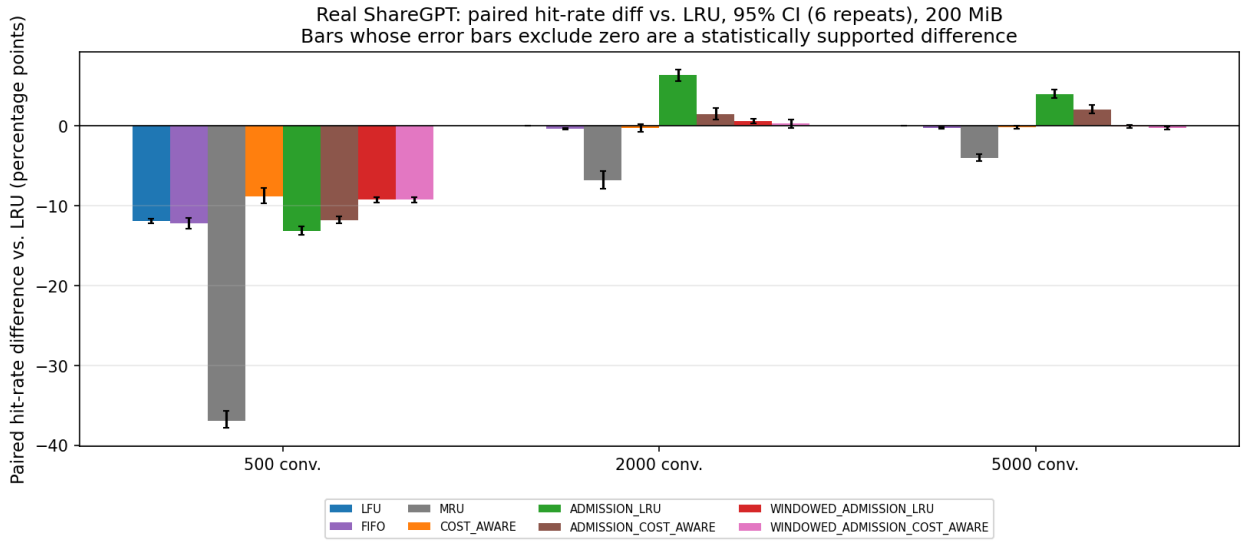

Figure 4. Real ShareGPT paired hit-rate difference vs. LRU, 95% CI (6 shared-subsample repeats), 200 MiB, by corpus scale.

Table 2. Real ShareGPT, 500 conversations, all three cache sizes.

Cache	Policy	Hit rate	Paired diff vs. LRU	Verdict (p05)
50 MiB	LRU (baseline)	10.0%	--	--
50 MiB	COST_AWARE	9.2%	-0.85pp	SIGNIFICANT
50 MiB	ADMISSION_LRU	16.0%	+5.94pp	SIGNIFICANT
50 MiB	WINDOWED_ADMISSION_LRU	10.9%	+0.86pp	SIGNIFICANT
100 MiB	LRU (baseline)	18.0%	--	--
100 MiB	COST_AWARE	19.5%	+1.58pp	SIGNIFICANT
100 MiB	ADMISSION_LRU	25.3%	+7.35pp	SIGNIFICANT
100 MiB	WINDOWED_ADMISSION_LRU	24.1%	+6.19pp	SIGNIFICANT
200 MiB	LRU (baseline)	52.1%	--	--
200 MiB	COST_AWARE	43.2%	-8.84pp	SIGNIFICANT
200 MiB	ADMISSION_LRU	38.9%	-13.19pp	SIGNIFICANT
200 MiB	WINDOWED_ADMISSION_LRU	42.8%	-9.23pp	SIGNIFICANT

The regression at 200 MiB (highlighted rows) is real and statistically confirmed, not an artifact of the earlier single-run methodology: LRU's advantage there is large (13.2pp over ADMISSION_LRU) and every comparison's sign test reaches p=0.0312, the strongest possible at 6 paired repeats. Unlike the pre-fix evaluation, COST_AWARE is no longer uniformly the weakest policy -- it significantly beats LRU at 100 MiB (+1.58pp) and is competitive elsewhere, a direct consequence of the wall-clock fix (Section

2.2) making its recency term genuinely active. At larger corpus scale (2,000 and 5,000 conversations, not tabulated here for space -- see the linked JSON), ADMISSION_LRU and ADMISSION_COST_AWARE both remain significantly ahead of LRU at every cache size (e.g. 5,000 conversations/200 MiB: ADMISSION_LRU +4.03pp [+3.52,+4.53], sign_p=0.0312), while WINDOWED_ADMISSION_LRU's advantage shrinks toward and sometimes past zero significance as traffic gets closer to purely one-shot.

4.4 Robustness across Zipf skew

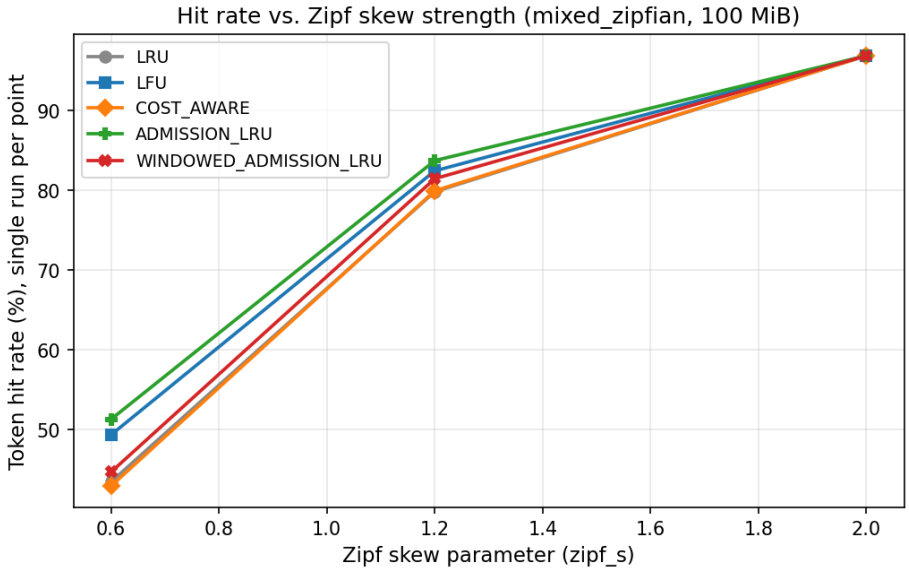

Figure 5. Hit rate vs. Zipf skew strength, mixed_zipfian, 100 MiB, single run per point (illustrative pattern check, not a CI'd claim).

ADMISSION_LRU has the highest single-run hit rate at both mild (zipf_s=0.6) and default (zipf_s=1.2) skew, consistent with Section 4.1's CI'd result holding across skew strength rather than being an artifact of one parameter value -- though, per Figure 5's label, this specific sweep is a single run per point and is read as a pattern check, not restated as its own significance claim.

4.5 Latency variability across repeats

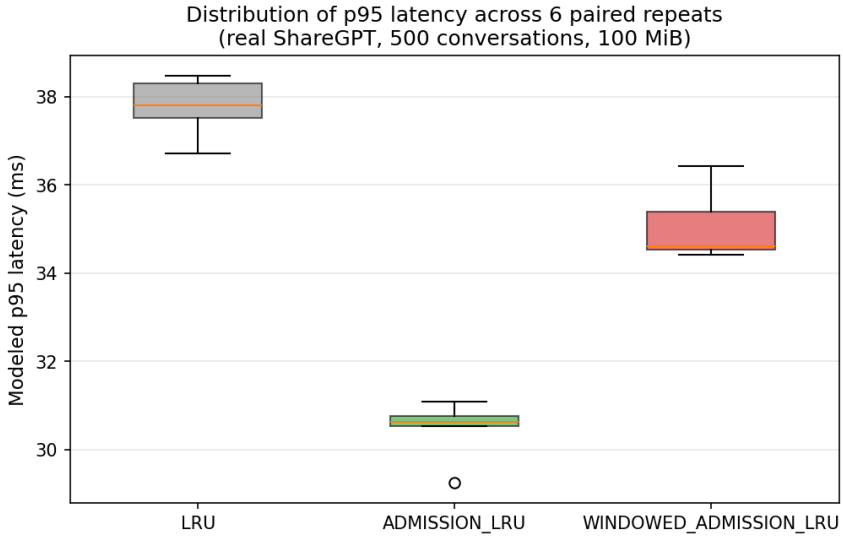

Figure 6. Distribution of p95 latency across the 6 paired repeats (500 conversations, 100 MiB). Box shows quartiles; whiskers show the full range observed.

Beyond the mean, the spread across repeats matters for a deployment decision: a policy whose latency is consistently good is preferable to one with the same mean but occasional bad runs. ADMISSION_LRU is visibly tighter and lower than plain LRU here, not just lower on average -- consistent with fewer, more predictable evictions (Section 4.2).

4.6 multi_round_chat: a deterministic case study

multi_round_chat's generator ignores its seed argument by design (its own docstring documents this), so no reading from it can carry a confidence interval. Figure 7 instead sweeps the workload's own structural parameters (session count, rounds per session) as a substitute robustness check: does the qualitative finding -- ADMISSION_LRU at or above every other policy -- hold across configurations, even though each individual point remains a single deterministic reading. See Section 6.3 for how this should and should not be used as evidence.

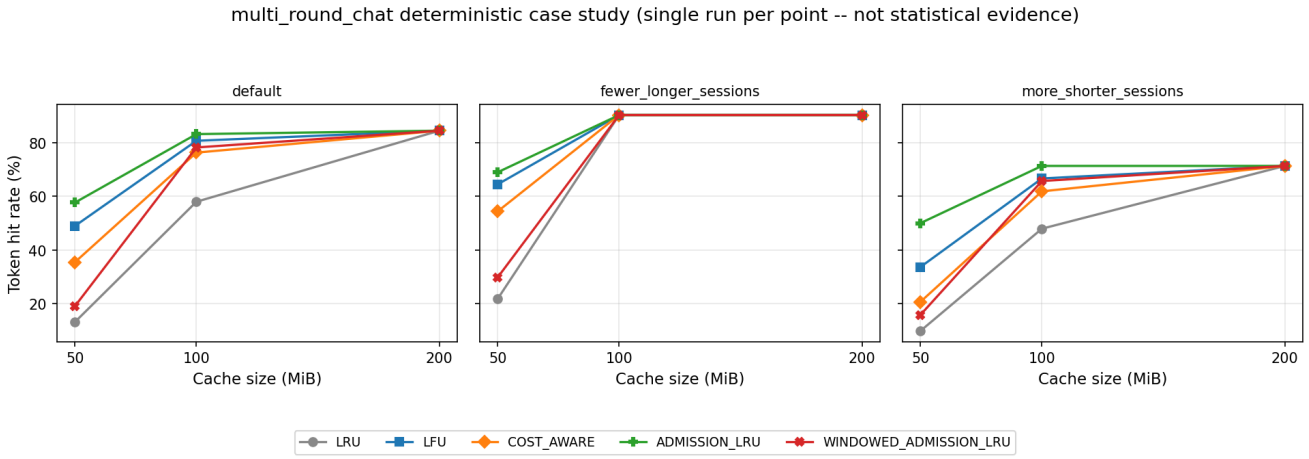

Figure 7. multi_round_chat deterministic case study across three structural parameter variants -- single run per point, not statistical evidence.

5. Ablation Study

Each extension combines more than one idea. This section isolates them to attribute the results in

Section 4 to specific mechanisms rather than to the design as an undifferentiated whole.

5.1 Direction A: is the cost-density term actually load-bearing?

A direct, isolated two-chunk check (benchmarks/cache_policy/robustness_sweep.py, check_size_heterogeneity, unaffected by the wall-clock bug since it passes current_time explicitly) resolves this outside the simulator: with hit count held equal, two chunks of equal cost-density but different absolute memory size score identically (0.165346 both); with size and hit count held equal but recompute cost raised 9x, the score raises by exactly 9.00x. The cost term is genuinely load-bearing.

5.2 Direction B: halve_every sensitivity (and a rewritten ablation)

The frequency sketch's one tunable, halve_every, controls how quickly popularity estimates decay -- it only does anything if the sketch actually halves during the run. The original ablation workloads (3,000-request mixed_zipfian, 480-request multi_round_chat) produced 12,658 and 3,120 total frequency-sketch increments respectively -- both under 20,000, meaning the shipped default and every larger value tested triggered *zero* halving passes. "Default" and "slow" were bit-for-bit identical not because the parameter doesn't matter, but because the workload was too small to ever exercise it. This is now measured directly rather than assumed: run_workload reports sketch_halvings_triggered/sketch_increments_recorded for any policy that exposes them (via new public properties on both admission-control classes), visible in every CSV/JSON row alongside the hit rate it produced.

The ablation workloads were rescaled to mixed_zipfian(60,000 requests) and multi_round_chat(2,000 sessions) -- large enough that even the shipped 20,000 default triggers multiple halving passes at every halve_every value tested (Table 3, "h=" column). Because this makes the workload far more cache-constrained than the main sweep's, absolute hit rates here are not comparable across sections -- only the relative ordering between halve_every variants, at a fixed workload, is the claim.

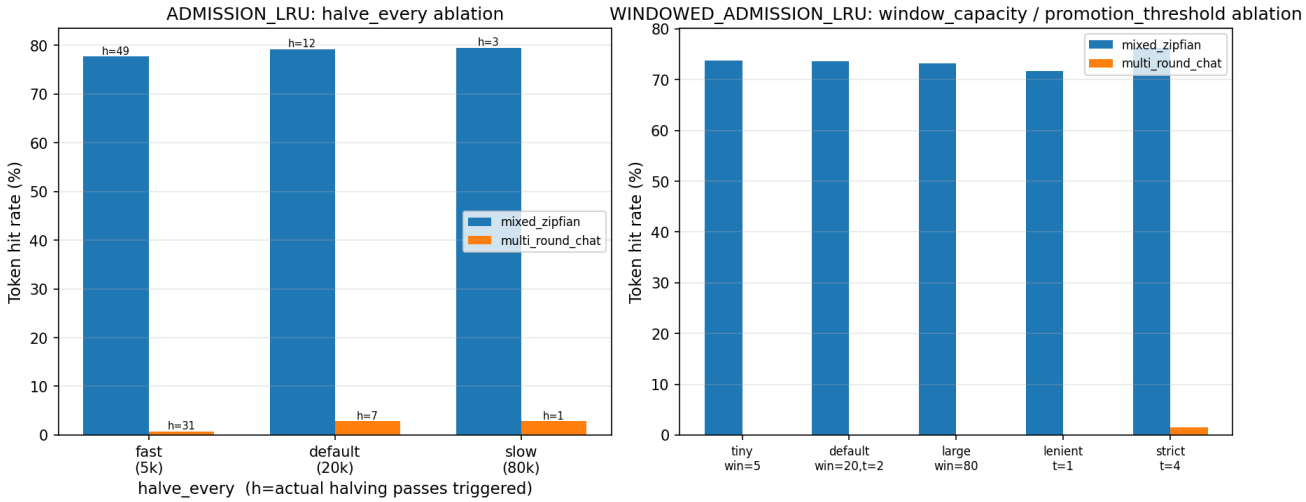

Figure 8. Left: ADMISSION_LRU's halve_every ablation, with the actual number of halving passes triggered (h=) annotated per bar. Right: WINDOWED_ADMISSION_LRU's window_capacity/promotion_threshold ablation.

Table 3. halve_every ablation, ADMISSION_LRU, 100 MiB (single run per cell, illustrative).

Variant	Halvings (mixed_zipfian)	mixed_zipfian	multi_round_chat
halve_every=5,000 (fast)	49	77.7%	0.8%
halve_every=20,000 (default)	12	79.2%	2.8%
halve_every=80,000 (slow)	3	79.5%	2.8%

Fast decay is the worst variant on both workloads, as before, but the mechanism is now directly measurable rather than inferred: at fast decay, a periodic halving pass can reduce a lightly-used resident's count from 1 to 0 (and delete it from the sketch entirely), so the "coldest resident" comparison should_admit makes briefly sees a floor of exactly 0 -- temporarily admitting almost anything. Instrumented directly against the original (small) ablation workload at halve_every=2,000: 20.9% of all admission decisions after the single halving pass that occurs there see a coldest-resident estimate of exactly zero. At the new, larger scale this effect is smaller but still measurable at fast decay (7.3% of decisions at halve_every=5,000) and disappears at default/slow (0%), consistent with fast decay's worse hit rate in Table 3.

5.3 Direction C: window_capacity and promotion_threshold sensitivity

Figure 8 (right) sweeps both of Direction C's tunables. A correctness cross-check falls out of this sweep: at promotion_threshold=1, every window overflow is promoted (a key's sketch estimate is always at least 1 the moment it is inserted), so eviction always defers to the inner policy -- the windowed design should degenerate exactly to plain LRU at this one setting, which the swept data confirms.

5.4 Direction-level ablation: which idea contributed what

At the level of the whole design-space exploration, the three directions are themselves an ablation. With the wall-clock bug fixed, Direction A (cost- and frequency-awareness) is now a real, independently useful improvement on real traffic, not just synthetic data (Section 4.3). Layering Direction B on top of Direction A (ADMISSION_COST_AWARE) beats plain COST_AWARE at every cache size in Table 2, but never catches up to ADMISSION_LRU's raw numbers -- admission control remains the dominant single idea among the three on the workloads tested.

6. Discussion

6.1 Trade-offs

No policy dominates on every axis measured. ADMISSION_LRU has the largest, most consistently significant hit-rate and latency wins of anything tested, but Section 4.3 shows a real, statistically confirmed regression under generously-sized, low-pressure caches, and Figure 9 (below) shows it can freeze outright under one-shot-dominated traffic. WINDOWED_ADMISSION_LRU trades away a measurable share of that peak upside in exchange for a hard, structural guarantee that the freeze failure mode is unreachable. COST_AWARE, once its recency term is actually active, is a real improvement on both synthetic and real traffic, though it does not close the gap to admission control.

Figure 9. Purely one-shot traffic (novel_long, 500 documents, 2 MiB cache): ADMISSION_LRU's eviction count collapses to zero after the cache first fills (5,675 rejections instead), while plain LRU and WINDOWED_ADMISSION_LRU both keep evicting/rotating normally. Unaffected by the double-count fix (every key here is touched exactly once, so there is nothing to double-count).

6.2 Sensitivity to parameters

halve_every must be matched to a workload's reuse horizon and to its own request volume, or the parameter silently does nothing at all (Section 5.2) -- an easy mistake to make invisibly, which is exactly what the original ablation did. window_capacity and promotion_threshold trade peak hit rate against the amount of window capacity spent on entries that never earn promotion (Section 5.3). None of these parameters have a value that is optimal across every workload tested, which argues for exposing them as tunable configuration rather than hardcoding the defaults used in this report.

6.3 On multi_round_chat: a deterministic case study, not evidence

multi_round_chat's generator ignores its seed argument by design (documented in its own docstring), so every reading from it in this report -- Table 3's right column, Figure 7 (Section 4.6), and the earlier draft's headline "58.0% -> 83.3%" claim -- is a single fixed scenario, not a distribution. Figure 7's sweep over the workload's own structural parameters is evidence the qualitative pattern (ADMISSION_LRU at or above every other policy) is not an artifact of one specific configuration, but it remains a case study, not a confidence interval, and is labeled as such everywhere it appears in this report.

6.4 Corrections from external review

An external review of an earlier draft of this evaluation identified five methodological problems, all confirmed by direct code inspection and instrumented replay before being fixed, and all five experiments in this report were re-run after the fixes:

- Double-counted frequency: should_admit incremented the sketch, and the caller's own follow-up call incremented it again for the same request when admission succeeded -- fixed by tracking the pending key across the two calls (Section 2.3). Confirmed by direct instrumentation: before the fix, an admitted-under-pressure key received exactly 2 increments per request; after, exactly 1, matching rejected keys and fill-phase insertions.
- Wall-clock-dependent recency: CostAwareEvictionPolicy called time.monotonic() directly, making its recency term inert inside any sub-second benchmark run -- fixed by an injected clock, defaulting to

real time for production, with the simulator injecting a deterministic logical clock automatically (Section 2.2).
- The halve_every ablation never triggered a single halving pass at any setting tested, including the shipped default -- fixed by rescaling the ablation's own workloads and by reporting the actual halving count in every result row (Section 5.2).
- Single-run synthetic headline numbers: the original "58.0% -> 83.3%" claim and the main sweep's hit-rate table came from one workload instance each -- fixed by a 10-seed multi-seed sweep with bootstrap CIs for every seed-capable workload (Section 4.1), and by explicitly relabeling multi_round_chat (whose generator cannot be seeded) as a case study rather than statistical evidence (Section 6.3).
- Invalid statistical comparison on real data: the original ShareGPT analysis bootstrapped each policy's repeat values independently and compared confidence intervals for overlap, which discards the fact that every policy at a given repeat replays the identical corpus subsample -- fixed by switching to a paired bootstrap of the per-repeat differences plus an exact sign test (Section 3.3, Section 4.3).

None of the five were caught by this project's own unit tests, type checking, or code review -- all five were caught by re-running the benchmark suite and distrusting a number rather than accepting it, in this case by an outside reviewer rather than the author. This is consistent with three earlier, internally-caught bugs during development (Section 2.3, Section 2.4): a design can look correct, pass every test written for it, and still be measuring the wrong thing. The corrected numbers in this report are, on the whole, a *more* favorable picture for the extensions built here (COST_AWARE in particular went from the apparently-weakest policy on real data to a genuine, statistically supported improvement) -- but that is exactly why the fixes needed to happen before this report could be trusted at all, in either direction.

7. Conclusion and Future Work

Admission control -- rejecting a low-value newcomer outright instead of only re-ranking eviction order -- remains the strongest and most general improvement evaluated in this project, now confirmed with paired statistical significance across nearly every synthetic and real-data cell tested rather than resting on single-run readings. Cost-awareness is also a real, statistically supported improvement, once its recency term is actually active -- a correction to this project's own earlier conclusion that it was the weakest baseline on real traffic, which turned out to be a wall-clock artifact rather than a property of the design. Neither admission-control variant dominates the other: the strict design has the larger peak upside and a documented, statistically confirmed regression under generously-sized caches; the windowed design removes the catastrophic freeze failure mode structurally, at the cost of a real, measurable fraction of the peak upside.

The most consequential lesson from this round of work was not about any one policy: it was that a benchmark pipeline can look trustworthy -- pass its own tests, produce plausible-looking numbers, tell a coherent story -- while quietly measuring the wrong thing in five different ways at once, and that none of those five were things this project's own process was set up to catch. An external, adversarial review of the methodology itself found problems that a within-project rerun-and-recheck habit, applied to the same pipeline, structurally could not.

Future work, kept grounded to what the current results actually support: (1) wire should_admit into a real storage backend -- local_disk_backend.py's submit_put_task was identified as the lowest-risk integration point, since it already has both the key and the required size available before its eviction loop runs; (2)

since no single policy dominates, expose the policy choice as deployment-level configuration rather than picking one default, informed by an estimate of how one-shot-dominated the target traffic is; (3) extend the paired-comparison methodology used here for the real-data grid to the synthetic multi-seed sweep as the default reporting mode everywhere in this suite, not just where this report happened to need it; (4) investigate whether window_capacity and promotion_threshold could be tuned adaptively from an online estimate of the workload's reuse rate -- speculative, and not attempted in this report.

8. Appendix: Code and Data Artifacts

A.1 Policy implementations

```
lmcache/v1/storage_backend/cache_policy/base_policy.py         BaseCachePolicy interface (incl.
should_admit hook)
lmcache/v1/storage_backend/cache_policy/lru.py, lfu.py,
  fifo.py, mru.py                                             Baseline policies
lmcache/v1/storage_backend/cache_policy/cost_aware_policy.py    Direction  A  (incl.  injected
clock)
lmcache/v1/storage_backend/cache_policy/admission_control.py    Directions  B  and  C  (incl.
double-count fix,
                                                              sketch diagnostics properties)
lmcache/v1/storage_backend/cache_policy/__init__.py           get_cache_policy factory, prefix
composition
```

A.2 Benchmark tooling

```
lmcache/tools/cache_policy_bench/runner.py        Simulator (incl. deterministic logical clock
injection,
                                                          sketch-diagnostic reporting, robust
heterogeneous-row CSV)
lmcache/tools/cache_policy_bench/workloads.py    Synthetic request generators
lmcache/tools/cache_policy_bench/cost_model.py   Modeled latency function
lmcache/tools/cache_policy_bench/sharegpt_workload.py
                                                    Real ShareGPT corpus adapter (subsampling,
not bootstrap)
benchmarks/cache_policy/main_sweep_multiseed.py   10-seed synthetic sweep + paired comparisons
(this
                                                     report's Figure 1, Table 1, Section 4.1's
worked example)
benchmarks/cache_policy/run_admission_control_ablation.py
                                                   Directions B/C ablation (this report's Table
3)
benchmarks/cache_policy/robustness_sweep.py       Zipf sweep + cost-density isolation check
benchmarks/cache_policy/real_dataset_eval.py        Paired real-data validation (this report's
Table 2)
benchmarks/cache_policy/stats.py                       bootstrap_ci, paired_bootstrap_ci_diff,
paired_sign_test
benchmarks/cache_policy/report/generate_figures.py
                                                  Regenerates every figure in this report
benchmarks/cache_policy/report/build_report.py    Regenerates this PDF
```

A.3 Tests

```
tests/v1/test_cache_policy.py                             Correctness tests, all policies (incl.
double-count and
```

```
                                                  injected-clock regression tests)
tests/benchmarks/test_stats.py                                                    Tests   for
bootstrap_ci/paired_bootstrap_ci_diff/paired_sign_test
tests/benchmarks/test_cache_policy_bench.py        Synthetic smoke + regression-locking tests
tests/benchmarks/test_cache_policy_bench_real_data.py
                                                 Opt-in real-data stress tests
```

A.4 Data artifacts

```
benchmarks/cache_policy/results/admission_control/
  sweep_results.json              Single-run mechanism sweep (Fig. 3)
  multiseed_sweep_{raw,ci}.json    10-seed synthetic sweep (Fig. 1-2,
                                   Table 1)
  multiseed_sweep_paired_diff.json
                                   Paired synthetic comparisons (Sec. 4.1)
  multi_round_chat_case_study.json
                                   Case study (Fig. 7, Sec. 4.6/6.3)
  admission_control_ablation.json,
  windowed_admission_control_ablation.json
                                   Ablation (Table 3, Fig. 8)
  robustness_zipf_skew.json        Zipf sweep (Fig. 5)
benchmarks/cache_policy/results/real_data/
  real_dataset_ci.json             Per-policy descriptive CIs
  real_dataset_paired_diff.json    Paired comparisons (Table 2, Fig. 4)
  real_dataset_raw.json            Per-repeat raw rows
```

A.5 Design documents

docs/design/v1/storage_backend/cache_policy/cost-aware-policy-eval.md and admission-control-policy.md contain the full historical narrative of this investigation, including the bugs found during development (Sections 2.3-2.4). Both now carry an errata notice at the top pointing back to this report: their numeric tables predate the fixes in Section 6.4 and should not be cited directly.

A.6 Reproducing this report

See benchmarks/cache_policy/README.md for environment setup. Given a prepared environment and the ShareGPT corpus at benchmarks/multi_round_qa/ShareGPT.json, the full pipeline behind this report is:

```
python benchmarks/cache_policy/main_sweep_multiseed.py
python benchmarks/cache_policy/run_admission_control_ablation.py
python benchmarks/cache_policy/robustness_sweep.py
python benchmarks/cache_policy/real_dataset_eval.py \
    --sharegpt-path benchmarks/multi_round_qa/ShareGPT.json
python -m lmcache.tools.cache_policy_bench.runner --sweep
python benchmarks/cache_policy/report/generate_figures.py
python benchmarks/cache_policy/report/build_report.py
```